Lecture 28.14: Decision Tree Regressor Implementation in Scikit-Learn

📌 Executive Summary

Full production pipeline for DecisionTreeRegressor, multi-dimensional tabular fitting, tuning max_depth and min_samples_leaf, and comparing against Linear Regression.

🏛️ Theoretical Formulation & Mathematical Architecture

💻 DecisionTreeRegressor API & Evaluation Framework

In Scikit-Learn, `DecisionTreeRegressor` implements regression tree induction with support for `criterion='squared_error'`, `'friedman_mse'`, `'absolute_error'`, and `'poisson'`.

Evaluation Metrics Formulation:

1. **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

2. **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

3. **Coefficient of Determination (R^2)**:

$$\boxed{R^2 = 1 - \frac{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{m}(y_i - \bar{y})^2} = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}}$$

Where $R^2 = 1.0$ represents a perfect regression fit, and $R^2 = 0.0$ corresponds to a naive baseline predicting the global mean $\bar{y}$.

💻 Vectorized Python Implementation

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Fit regularized regressor
reg_model = DecisionTreeRegressor(max_depth=4, min_samples_leaf=5, random_state=42
reg_model.fit(X_1d, y_1d)

preds = reg_model.predict(X_1d)
mse = mean_squared_error(y_1d, preds)
r2 = r2_score(y_1d, preds)

print(f"Regression MSE: {mse:.4f}")
print(f"Regression R^2: {r2:.4f}")
assert r2 > 0.80, "Depth 4 tree must capture non-linear sine trend with high R^2"
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.